



CMP6230 Data Management and MLOps

E-Commerce Behavior Pipeline

Final Individual Report Submission

Student Name: Shekhar Lamichhane Magar

Student Number: 23189647

Programme: BSc (Hons) Computer and Data Science

Module: CMP6230 Data Management and MLOps

Module Coordinator: Rupak Koirala

Submission Date: 17th May 2026

Email: shekhar_23189647@sunway.edu.np

Word Count: $\approx$ 4,000

Abstract

This report outlines the design and implementation of a five stage MLOps pipeline for the purpose of CMP6230 (Data Management and ML Operations). The **E-Commerce Behavior Dataset** (8000 users) that had been tested on practical and technical aspects against the two other Kaggle datasets was selected for the binary classification problem: whether a user will click on an advertisement in a browsing session or not. This pipeline includes data ingestion with schema validation, ELT preprocessing and feature engineering (transformations after data is loaded), 3-algorithm model comparison with 5-fold cross validation, deployment on REST API using FastAPI and Docker, and monitoring data drift using Evidently AI. All stages are orchestrated by an Apache Airflow DAG and connected via an in-memory data bus called Redis. The storage system is designed on the basis of a **One Big Table (OBT)** design on MariaDB ColumnStore. Random Forest classifier got the highest ROC-AUC score (0.51), Yet the three models ended up with almost random performance levels this being due to the synthetic nature of the dataset rather than limitations of the pipeline itself. Compliance with GDPR and security concerns have also been mentioned.

Contents

1	Introduction	6
2	Dataset Selection and Description	6
2.1	Candidate Dataset Evaluation	6
2.2	Dataset Characteristics and Logical Structure	7
2.3	Class Distribution and Research Questions	8
3	Initial Data Storage Plan and Pipeline Architecture	9
3.1	Data Storage Architecture: One Big Table (OBT)	9
3.2	Data Pipeline Strategy: Extract-Load-Transform (ELT)	11
3.3	Overall Pipeline Architecture	12
4	Final MLOps Pipeline Plan	13
4.1	Data Ingestion	14
4.2	Data Preprocessing	15
4.3	Hyperparameter Tuning and Model Training	16
4.4	Model Deployment	16
4.5	Workflow Orchestration with Apache Airflow	17
5	Final Pipeline Implementation and Model Deployment	18
5.1	Infrastructure and Containerisation	18
5.2	Model Serving with FastAPI	19
5.3	Experiment Tracking with MLflow	19
5.4	Reproducibility	20
6	Data Analysis and Insight Generation	20
7	Pipeline Evaluation and Monitoring	24
7.1	Pipeline-Level Evaluation	24
7.2	Data Drift Detection with Evidently AI	26
8	Evaluation of Wider Issues	27
8.1	Key Definitions	27
8.2	GDPR and Its Impact on Data-Driven Systems	28
8.3	Security and Privacy in the Current Pipeline	28
9	Conclusion, Recommendations, and Future Work	29
9.1	Personal Reflection	29
9.2	Conclusion	30
9.3	Recommendations and Future Work	31
	References	32

List of Figures

1	Logical structure (ERD) of the E-Commerce Behavior Dataset source file. The single flat table has no foreign keys; all 10 columns belong to one entity.	8
2	Complete MLOps pipeline from the CSV source to MariaDB ColumnStore (ELT load) First, Airflow organizes a five-stage DAG (ingest → preprocess → tune → train → evaluate). Then, Redis transfers data among the steps. MLflow keeps a record of the experiments. FastAPI makes the predictions available, and Evidently AI performs ongoing drift monitoring of the entire system lifecycle.	13
3	Airflow DAG graph view showing the five pipeline tasks executing sequentially. Drift monitoring runs as a separate continuous schedule, not as a terminal DAG step.	18
4	Docker Compose launching the three infrastructure containers.	18
5	FastAPI Swagger UI demonstrating the prediction endpoint.	19
6	MLflow experiment tracking showing model metrics comparison across multiple runs.	19
7	Missing values distribution: 2% per column, uniform across classes, justifying median imputation [Pedregosa et al., 2011].	21
8	Target class distribution: 49.8% No vs. 50.2% Yes — near-perfectly balanced, ruling out class imbalance as a concern.	21
9	Feature histograms: most continuous predictors show uniform distributions, a hallmark of synthetic data.	22
10	Box plots: identical distributions across both classes, confirming no class-discriminating signal in numeric predictors [He et al., 2014].	22
11	Categorical distributions: near-identical 50/50 splits across all categories, confirming no class-conditional signal [Richardson et al., 2007].	23
12	Correlation matrix: near-zero correlations (−0.02 to +0.01) between <code>ad_clicked</code> and all features, confirming no linear relationship.	23
13	Confusion matrix for the best Random Forest model on the held-out test set. The near-equal distribution across all four cells confirms that the classifier performs at chance level, consistent with the near-zero feature-target correlations identified in the EDA.	25
14	ROC curve for the Random Forest classifier (AUC = 0.51). The curve hugs the diagonal, confirming near-random discriminative power [Fawcett, 2006].	25
15	Precision-Recall curve (AP = 0.52). The flat precision line confirms the model has no ability to rank positive instances above negative ones [Fawcett, 2006].	25

16	Random Forest feature importances ranked by mean decrease in impurity. <code>engagement_score</code> , <code>time_on_site</code> , and <code>pages_viewed</code> receive the highest scores, reflecting their higher variance rather than genuine predictive signal — a known artefact of impurity-based importance in datasets with no true class-discriminating features [Pedregosa et al., 2011].	26
17	Evidently AI monitoring dashboard displaying data drift metrics over successive pipeline runs.	27

List of Tables

1	Dataset candidates considered for the project.	7
2	Feature summary of the E-Commerce Behavior Dataset (9 predictors + 1 target).	8
3	Storage strategy comparison evaluated at the planning stage.	10
4	OBT schema definition in MariaDB ColumnStore (user_session_obt). . .	11
5	Summary of tools and technologies used in the ML pipeline.	14
6	Three-algorithm comparison (5-fold CV with RandomizedSearchCV). . .	24
7	Monitoring metrics and alert thresholds.	27
8	Security improvement recommendations for production deployment. . .	29

1 Introduction

Predicting the click-through-rate (CTR) is one of the main problems in digital marketing. Understanding if the user will click on the ad can be a great help to the marketers - they can put an ad where it is most likely to be viewed and clicked, avoid spending money on ads that are not getting the results, and show users content that they are interested in proceeding [He et al., 2014]. On the data engineering side, the problem is just as interesting: it needs the processing of demographic, behavioural, and transactional features data diversity within a reproducible and observable pipeline.

Background

MLOps incorporates the ideas of DevOps throughout the entire lifecycle of machine learning systems [Sculley et al., 2015]. Initial ML rolls-out kept building up technical debt without being aware due to versioning, monitoring, and data pipeline without reproducible features enable. The software like Apache Airflow [Apache Software Foundation, 2024] and MLflow [MLflow, 2024] solve these issues by workflow orchestration and experiment tracking respectively.

Problem Statement

The primary machine learning question that is being addressed is a binary classification problem: given features of a user session - such as age gender device type, time on site, pages viewed, previous purchases, items in cart, and discount exposure - predict whether the user clicked on an advertisement (`ad_clicked = 1`) or not (`ad_clicked = 0`). ROC-AUC was selected as the main evaluation metric, as it reflects the model's ability to distinguish classes at different thresholds and it is also fairly robust to small class imbalances [Fawcett, 2006].

2 Dataset Selection and Description

2.1 Candidate Dataset Evaluation

Before selection, three candidate datasets were evaluated, for the clarity of the target variable, feature diversity, size and suitability for a real-world CTR classification problem. The comparison is summarized in Table 1.

Table 1: Dataset candidates considered for the project.

Dataset	Rows	Task Type	Target	Decision / Key Reason
AI Impact on Job Market 2024–2030 [sahilislam007, 2024]	~500	Multi-class classification	Ordinal / subjective	Rejected — too small; target is expert-derived and subjective
E-Commerce Behavior Dataset [asifxzaman, 2024]	8,000	Binary classification	Pre-labelled (0/1)	Selected — clear business value; clean binary label; diverse features
US Tariff & Trade War Dataset [belbino, 2024]	2,500+	Time-series regression	None (derived)	Rejected — no pre-defined label; temporal leakage risk

E-commerce Behavior Dataset is chosen as the primary source of data due to the following reasons: first, the predictions concerning click-through rate are obviously of practical significance and this will provide a significant increase in the profitability of advertisement campaigns due to the possibility of identifying the most interested users. Moreover, the dataset possesses almost equal class distribution, a relatively large number of behavioral and demographic attributes, as well as pre-classified examples, which will provide excellent grounds for successful supervised learning.

2.2 Dataset Characteristics and Logical Structure

The E-Commerce Behavioral Dataset [asifxzaman, 2024] is a collection of behavioral data from sessions. E-Commerce Behavior Dataset provides session-level behavioral data. The 8,000 ecommerce users that were made public in Kaggle. The dataset represents the user demographics are integrated into the session level and realistic digital retail interactions are achieved. (age, gender), transactional (time on site, pages viewed, device type), and session activities (time on site, pages viewed, device type). The social media information that's visible to both others and them (prior purchases, cart items, discount visibility). The outcome of an ad interaction is a binary variable (`ad_clicked`).

The file is a single, flat CSV file with 10 columns and 8,000 rows (about 300 kB). The logical structure of the original data is depicted in Figure 1 in the form of entity diagram. No one-to-one or many-to-many relationships - all columns are in one entity.

USER_SESSION		
user_id (PK)	INT	Unique session ID
age	INT	User age
gender	VARCHAR	Male/Female/Other
device_type	VARCHAR	Desktop/Mobile/Tablet
time_on_site	FLOAT	Minutes on site
pages_viewed	INT	Number of pages
previous_purchases	INT	Historical purchases
cart_items	INT	Items in cart
discount_seen	BOOLEAN	0/1 discount shown
ad_clicked (target)	BOOLEAN	0/1 click or not

Figure 1: Logical structure (ERD) of the E-Commerce Behavior Dataset source file. The single flat table has no foreign keys; all 10 columns belong to one entity.

Table 2 describes all columns in the dataset.

Table 2: Feature summary of the E-Commerce Behavior Dataset (9 predictors + 1 target).

Column	Type	Role	Description
user_id	Integer	ID	Unique session identifier (dropped before training)
age	Integer	Predictor	User age in years (ratio scale)
gender	Varchar	Predictor	Self-reported gender (nominal; OHE applied)
device_type	Varchar	Predictor	Desktop / Mobile / Tablet (nominal; OHE applied)
time_on_site	Float	Predictor	Session duration in minutes (ratio scale)
pages_viewed	Integer	Predictor	Pages visited during session (ratio scale)
previous_purchases	Integer	Predictor	Historical purchase count (ratio scale)
cart_items	Integer	Predictor	Items in cart at session time (ratio scale)
discount_seen	Boolean	Predictor	Discount banner shown (0/1)
ad_clicked	Boolean	Target	Ad clicked binary label (0/1)

2.3 Class Distribution and Research Questions

The dataset is almost perfectly balanced: 49.8% No vs. 50.2% Yes for `ad_clicked`. Because of this, the effect of class imbalance is discarded as a reason of model failure and plain accuracy is a complementary measure along ROC-AUC and F1-Score. Since this is a baseline experiment, no SMOTE or class weighting is needed here.

Three data analysis research questions were designed to direct the EDA:

- RQ1.** Which session features (time on site, pages viewed, cart items) show the most statistically distinguishable difference between users who clicked and those who did not?
- RQ2.** Are missing values distributed uniformly across both classes, or do they concentrate in one class?
- RQ3.** Do contextual features (device type, discount seen) carry class-conditional distributions that provide independent predictive signal beyond demographic features?

3 Initial Data Storage Plan and Pipeline Architecture

3.1 Data Storage Architecture: One Big Table (OBT)

Before setting up the storage layer, I evaluated three options. A Star Schema is perfect for OLAP reporting but my source is just a single flat CSV file with no relational structure - normalising it would have only increased the join overhead without any benefit. I also thought about a normalised 3NF design but since my access pattern is mainly read-heavy, a columnar store will be more efficient. MariaDB ColumnStore was the obvious choice.

Machine Learning Needs Flat Tables

ML algorithms don't understand joins. They expect one row per observation, one column per feature. OBT delivers this directly:

```
SELECT age, gender_Male, time_on_site, pages_viewed,  
       cart_items, engagement_score, ad_clicked  
FROM user_session_obt;
```

A Star Schema would require a multi-table JOIN every time - more code, execution takes more time, and no benefit for this single-source dataset.

The Data Is Already Flat

This dataset is not relational. Each row represents a whole user session and all its attributes. Normalising it into multiple tables would be like taking something simple. Adding to that, and making it complicated, would be to then put it all back together for each query.

ColumnStore Actually Prefers OBT

MariaDB ColumnStore is a columnar database. When using OBT, only the columns are read and requested from disk. JOINS cause row re-materialisation and lose data with a Star Schema. the columnar advantage. For a pipeline that is going to repeat it's

SQL scan of all 8,000 rows, The cumulative effect of performance difference is always possible.

Storage Overhead Is Tiny

There is some redundancy in OBT, but 8,000 rows x 15 columns is 300KB uncompressed. (under 100KB compressed). The time saved in not having to implement a Star Schema went to Rather, they should go straight to feature engineering.

Table 3: Storage strategy comparison evaluated at the planning stage.

Strategy	Pros	Cons	Decision
OBT (ColumnStore)	No joins; single query for ML; columnar scans maximally efficient; zero JOIN re-materialisation cost	Minor redundancy; harder to extend with entity types	Chosen
Star Schema	Clean fact/dim separation; suits OLAP reporting	JOIN overhead defeats columnar advantage; more code; no benefit for single-source flat data	Rejected
Snowflake Schema	More normalised; less redundancy	Multi-level joins; complex queries; no benefit for this dataset	Rejected
Normalised (3NF)	Full referential integrity; zero redundancy	Multiple joins needed; worst pattern for columnar engine [Zaitsev et al., 2021]	Rejected

The Final OBT Schema

Table 4: OBT schema definition in MariaDB ColumnStore (user_session_obt).

Column	Type	Description
user_id	INT	Unique session identifier (primary key)
age	INT	User age in years (raw; scaled before training)
gender_Male	TINYINT	1 if Male, 0 otherwise (one-hot encoded)
gender_Female	TINYINT	1 if Female, 0 otherwise
device_Desktop	TINYINT	1 if Desktop, 0 otherwise
device_Mobile	TINYINT	1 if Mobile, 0 otherwise
device_Tablet	TINYINT	1 if Tablet, 0 otherwise
time_on_site	FLOAT	Session duration in minutes
pages_viewed	INT	Number of pages visited
previous_purchases	INT	Historical purchase count
cart_items	INT	Items in cart
discount_seen	TINYINT	1 if discount banner shown
engagement_score	FLOAT	$(\text{pages_viewed} \times \text{time_on_site}) / (\text{cart_items} + 1)$
ad_clicked	TINYINT	Target variable (0 or 1)
pipeline_run_id	VARCHAR(50)	Tracks which pipeline run created this row
ingestion_ts	DATETIME	When the row was loaded

3.2 Data Pipeline Strategy: Extract-Load-Transform (ELT)

Why ELT Over ETL For This Project

The main reason for choosing ELT over ETL was data preservation and reproducibility. In ELT, raw data is loaded into MariaDB ColumnStore first, and then transformed. are applied. This will ensure that the original Kaggle CSV is downloaded in exactly the same format, generating the immutable source of truth.

Complex stateful operations such as one-hot encoding, StandardScaler, feature engineering, etc. and stratified splitting are still done in Python, although they occur after the data. is already safely stored in the database. There are three benefits to this:

1. **Auditability:** The raw data cannot be changed. If a bug is found in the pre-processing, the code can be fixed and run again on the original stored data without downloading it again from Kaggle.
2. **Reproducibility:** Every experiment begins with the exact same raw data. Different preprocessing strategies can be compared fairly.
3. **Separation of concerns:** Ingestion loads the data. Preprocessing only transforms it.

Neither step interferes with the other.

ELT is also the standard pattern used in the industry for multi terabyte or streaming sources. Via BigQuery, Snowflake, Redshift. The use of ELT here creates habits which transfer to production environments.

The Data Is Already Clean

Synthetic datasets are typically very clean and this is no exception. Missing Across the entire data set, values are about 2% Nothing structural - Median imputation is ok. There's no bad data, no inconsistent date formats, malformed rows. Categorical columns like `gender` and `device_type` is standardised as is; there are no typos or capitalisations issues. Target is unambiguously labelled 0 or 1.

Because the data barely needs cleaning before it is usable, running heavy preprocessing before loading felt like overkill. ELT lets me store the raw data first and apply transformations later, on demand.

How ELT Actually Runs in This Pipeline

1. **Extract:** The Kaggle API downloads the CSV using credentials stored in `.env`.
2. **Load:** Raw data goes straight into MariaDB ColumnStore as a staging table. Missing values stay missing. Nothing gets changed yet.
3. **Transform:** After the load completes successfully, `preprocessing.py` reads from the database, applies all transformations (imputation, encoding, scaling, feature engineering), and writes the clean feature matrix to a separate table for model training.

The raw data stays untouched. If a different preprocessing approach is needed later, only the transform step needs to be re-run.

3.3 Overall Pipeline Architecture

The entire pipeline from data source to the prediction API and the monitoring dashboard is shown in Figure 2. The pipeline consists of 5 logical stages, each implemented as a separate Python module.

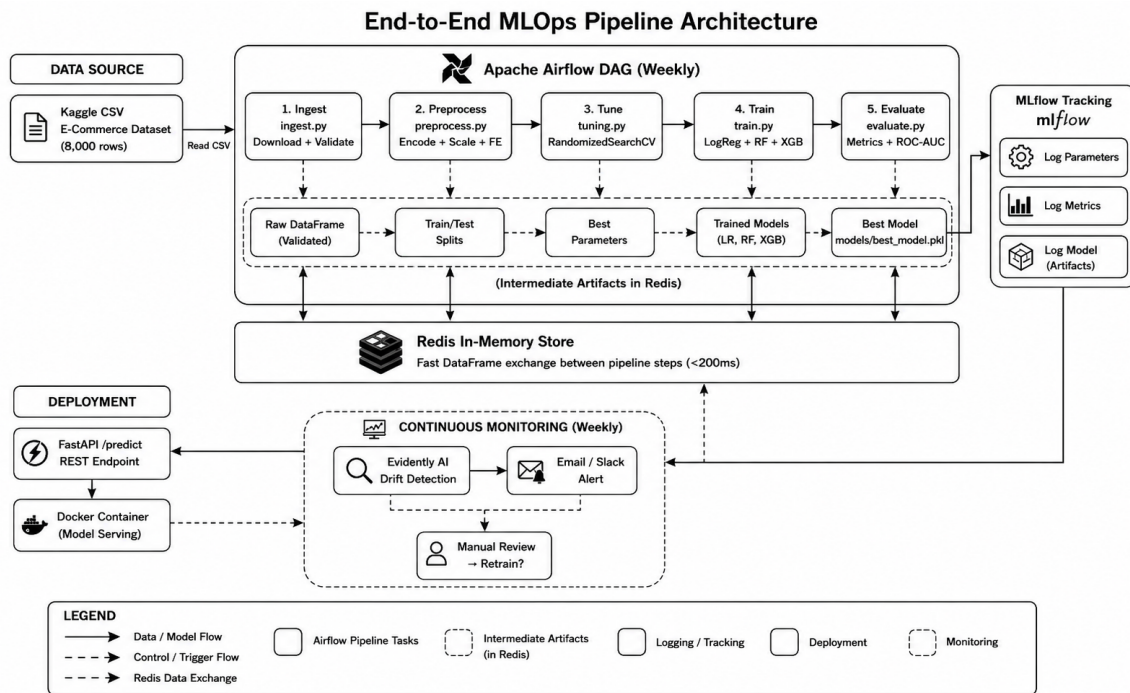


Figure 2: Complete MLOps pipeline from the CSV source to MariaDB ColumnStore (ELT load) First, Airflow organizes a five-stage DAG (ingest → preprocess → tune → train → evaluate). Then, Redis transfers data among the steps. MLflow keeps a record of the experiments. FastAPI makes the predictions available, and Evidently AI performs ongoing drift monitoring of the entire system lifecycle.

4 Final MLOps Pipeline Plan

This section presents the final pipeline design across five stages and the roles involved at each stage. Each stage is structured using the **What / Why / How / Who** framework.

Table 5 summarises the tools and technologies used across all stages.

Table 5: Summary of tools and technologies used in the ML pipeline.

Component	Tool / Library	Purpose
Data ingestion	Kaggle API + python-dotenv	Repeatable, credential-safe download
Schema validation	Great Expectations	Three checkpoint rules before any row is committed
Data processing	pandas + scikit-learn	OHE, StandardScaler, interaction terms, stratified split
Feature engineering	Custom Python	engagement_score + age × discount_seen
Inter-step transfer	Redis	In-memory DataFrame bus between Airflow tasks (<200ms)
Model training	scikit-learn + XGBoost	LogReg, RF, XGBoost with RandomizedSearchCV
Experiment tracking	MLflow	Parameters, metrics, artefacts, Model Registry
EDA / plots	Matplotlib + Seaborn + SHAP	Visualisations and feature importance
Orchestration	Apache Airflow	DAG-based pipeline scheduling
Deployment	FastAPI + Docker	Containerised REST API
Monitoring	Evidently AI	PSI drift detection + retraining triggers
Storage	MariaDB ColumnStore	Columnar data warehouse for ELT output

4.1 Data Ingestion

What: Data ingestion is the first step of the data pipeline. It gets the fresh CSV from Kaggle, performs schema validation and inserts the validated rows into MariaDB ColumnStore. Also, it converts the DataFrame into Redis for quick downstream usage.

Why: Recording raw data before any transformation provides an auditable, reproducible source. Schema validation at the boundary prohibits faulty or out-of-range values from being sent downstream to the feature table, which is a frequent cause of unnoticed model degradation in production pipelines. Redis reduces the time of inter-step transfer to under 200 milliseconds, versus several seconds for disk-based CSV handoffs.

How: The module (`src/data_ingestion.py`) reads Kaggle credentials from `.env` file (which is kept out of Git via `.gitignore`), downloads the CSV, and executes a Great Expectations [Great Expectations, 2024] checkpoint that validates three schema rules: (1) `user_id` field is unique in the dataset; (2) the only values in `ad_clicked` column

are 0 or 1; (3) the values in `age` column are within 13 and 100 inclusive. Each row failing a check is quarantined instead of removed. The rows passing validation are inserted in one go into the OLTP MariaDB staging table `raw_user_sessions`. The raw DataFrame is simultaneously serialised into Redis. The `pipeline_run_id` and `ingestion_ts` introduced at this step give the whole historical record.

Who: A Data Engineer is the one who sets up the database connections, checks the row counts after ingestion for accuracy, and keeps the Docker Compose infrastructure running. Data Scientists get their hands on the stored data either via Redis or by making direct MariaDB queries.

4.2 Data Preprocessing

What: Initially, the preprocessing module (`src/preprocessing.py`) cleans the raw data by removing the extraneous parts. After that, it transforms categorical variables into numerical format, normalizes numerical features, creates new features and, in the end, performs stratified train/test splits.

Why: A classifier is not capable of directly consuming raw data. Categorical columns need to be encoded, and numeric features within different scales would bias distance-based algorithms if an adequate scaling was not used. Also, a stratified split guarantees that the binary target maintains the same proportion of classes in both training and testing, which is a prerequisite for unbiased evaluation [[Pedregosa et al., 2011](#)].

How: The following transforms are applied in order:

1. Remove the user id non-feature `user_id`.
2. Fill the gap with the numeric values using the median.
3. One-hot encode `gender` and `device_type` as they are nominal with no ordinal relationship.
4. Use `StandardScaler` on `age`, `time_on_site`, `pages_viewed`, `previous_purchases`, `cart_items`. `StandardScaler` was chosen over `MinMaxScaler` since the features are not bounded. `MinMaxScaler` would have scaled the outliers into the range floor whereas `StandardScaler` would have left them unchanged.
5. Derive the composite feature:

$$\text{engagement_score} = \frac{\text{pages_viewed} \times \text{time_on_site}}{\text{cart_items} + 1} \quad (1)$$

This captures browsing intensity relative to cart state, acting as a proxy for purchase intent.

6. Use interaction term between `age` and `discount_seen`, to verify whether younger users respond differently to promotional banners.

7. Apply a stratified 80/20 train/test split with fixed random seed (42) for reproducibility [[Scikit-learn Developers, 2018](#)].

The processed splits are converted into sequences that are stored into Redis that is an in-memory data communication channel between the pipeline stages, which not only dispels all the file-path issues but also eliminates the I/O latency linked to the old CSV-based handoff method.

Who: A Data Scientist designs and validates the preprocessing logic; a Data Engineer implements it as an Airflow pipeline task.

4.3 Hyperparameter Tuning and Model Training

What: Three classification models are trained and their results are compared. These include Logistic Regression that acts as a linear reference model, Random Forest as a non-linear combination of decision trees, and XGBoost based on gradient boosting method. To perform the hyperparameter optimization, RandomizedSearchCV and 5-fold stratified cross-validation is employed. The best model with ROC-AUC is saved in MLflow.

Why: A single model cannot provide a valid reason for declaring it the best choice. By comparing the three models, we get the evidence to support model selection and make the final choice justifiable. Because of its ability to explore the search space in a large way while pruning less promising configurations, RandomizedSearchCV was selected over GridSearchCV because it is more sample-efficient - performs a broad search of the space and at the same time removes poor configurations early, reducing total computing [[Pedregosa et al., 2011](#)]

We chose ROC-AUC as the primary evaluation metric for selection. The reason is that it measures the quality of the discrimination between the classes for all possible classification thresholds. Besides, it is robust to slight class imbalance [[Fawcett, 2006](#)].

How: The script `src/train_model.py` obtains splits from Redis, executes 5-fold stratified cross-validation for each algorithm, tunes each one via RandomizedSearchCV, logs all runs to MLflow [[MLflow, 2024](#)], and saves the top model based on ROC-AUC to `models/best_model.pkl`. Evaluation is based on Accuracy Precision Recall, F1-Score, and ROC-AUC.

Who: A Data Scientist defines the search space and validates CV results. The pipeline automates execution via the Airflow DAG.

4.4 Model Deployment

What: The MLflow-registered model is deployed as a production REST API endpoint with FastAPI [[Ramírez, 2024](#)] and containerized by Docker [[Docker Inc., 2024](#)]. The

container makes `POST /predict` available, which takes user session data in JSON format and replies with `{prediction, probability, model_version}`.

Why: FastAPI was selected instead of Flask because it offers native async support, automatic generation of OpenAPI docs, and Pydantic validation of request bodies - this way, wrongly formed input to inference is caught at the API boundary, not after it reaches the model. Docker ensures that the inference environment is exactly the same at the level of bytes between development, staging, and production, thereby removing the dependency version mismatches that pretty often lead to failure of deployed ML models

How: The service (`api/app.py`) loaded the trained model from `models/best_model.pkl`, takes in the JSON input and maps it to the model's expected feature vector, and outputs the result with accompanying class probability and model version information. We validate the response using Pydantic models and any invalid request will not reach the classifier.

Who: A DevOps or ML engineer manages the container and deployment environment. Downstream consumers (e.g. a digital marketing team) call the API to make real-time ad targeting decisions.

4.5 Workflow Orchestration with Apache Airflow

What: Apache Airflow [[Apache Software Foundation, 2024](#)] orchestrates the core pipeline stages as a Directed Acyclic Graph (DAG) defined in `dags/ecommerce_pipeline.py`: `ingest_data → preprocess_data → tune_hyperparameters → train_model → evaluate_model`.

Why: Airflow automates the process of executing steps in the correct order, which otherwise, without orchestration would mean manually doing each step one after another. Besides that, it also offers a web interface to track task state and supports retrying tasks after failures.

How: Every DAG node runs as a `PythonOperator` with imports deferred so that heavy libraries are not loaded during DAG parsing time. All tasks are configured with `do_xcom_push=False` since data is passed between stages through Redis and not XCom. Also, `catchup=False` stops Airflow from performing backfill for missed runs.

Who: A Data Engineer or MLOps Engineer is responsible for defining the DAG, monitoring task states via the Airflow web UI, and handling any failed task retries. Data Scientists interact with Airflow indirectly through scheduled pipeline runs.

Drift monitoring is considered as a continuous operational activity, not a single-stage event. The `src/drift_detection.py` script is scheduled independently on a weekly basis, running comparison of live traffic distributions against training data and sending a retraining DAG run request when drift levels exceed the predefined limits. This is an accurate representation of the monitoring concept: it is a continuing duty that is

performed alongside, and not after, the main pipeline.ot sequentially after, the main pipeline.

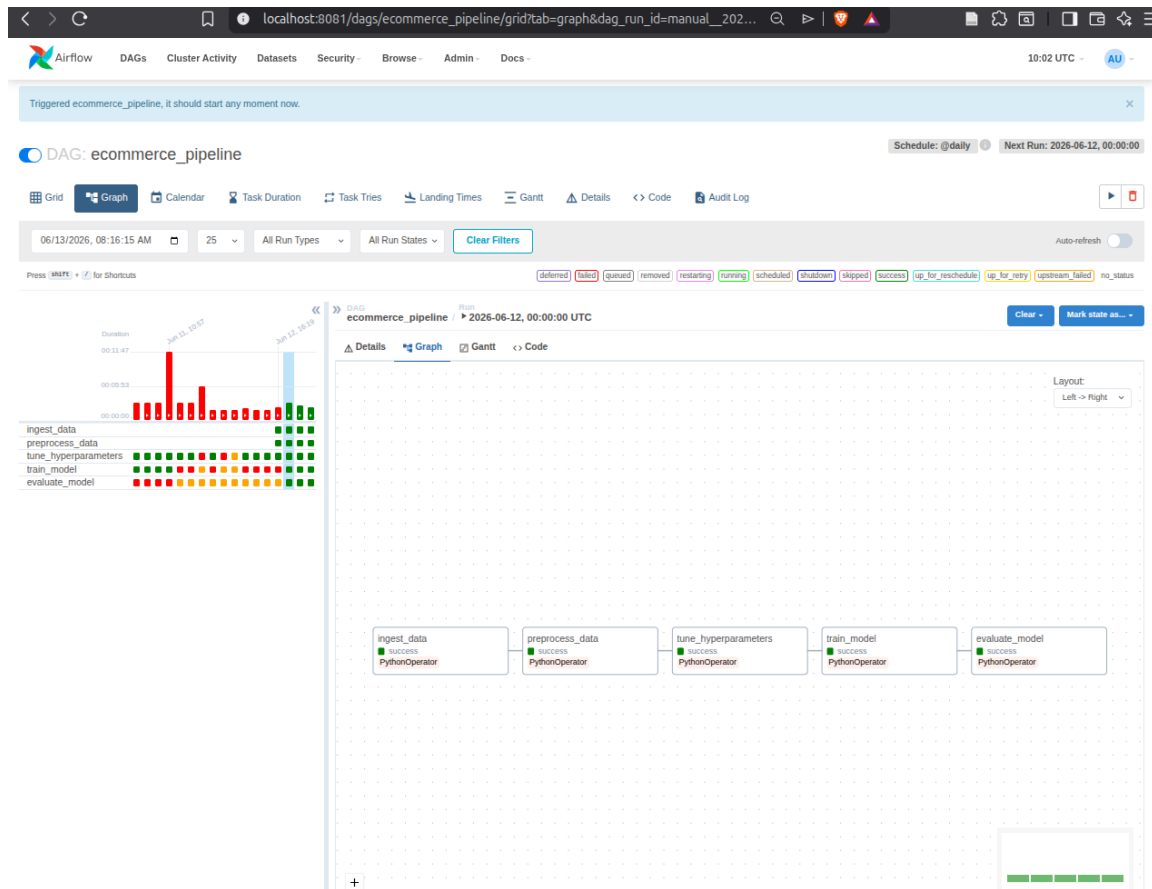


Figure 3: Airflow DAG graph view showing the five pipeline tasks executing sequentially. Drift monitoring runs as a separate continuous schedule, not as a terminal DAG step.

5 Final Pipeline Implementation and Model Deployment

5.1 Infrastructure and Containerisation

All the infrastructure pieces are set up by Docker Compose. The three containers that are made available are `mariadb_server` (MariaDB ColumnStore), `redis_store` (Redis), and `mlflow_server` (MLflow tracking server). Since we are using containers, this stack can be easily moved and executed on any machine that has Docker installed.

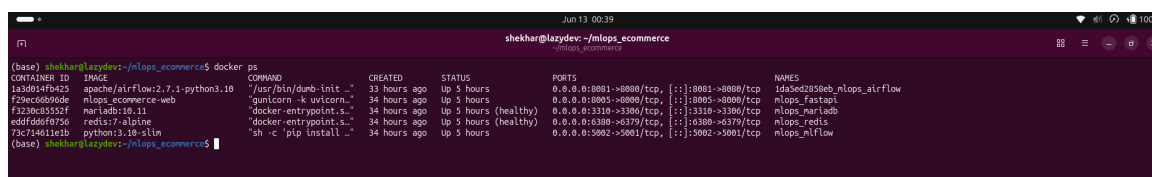


Figure 4: Docker Compose launching the three infrastructure containers.

5.2 Model Serving with FastAPI

The FastAPI service (`api/app.py`) makes a POST `/predictroute` that accepts user session attributes in JSON format and sends the predicted class and probability back as a response. Figure 5 displays the automatic Swagger UI.

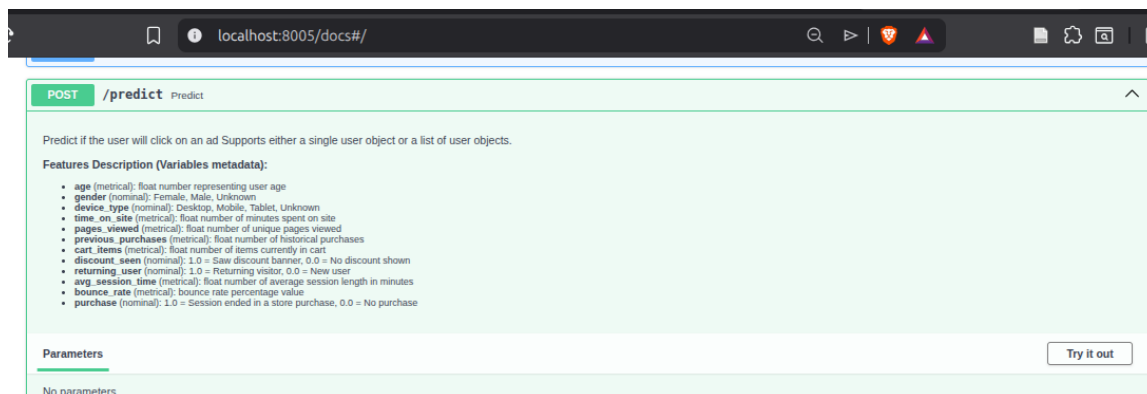


Figure 5: FastAPI Swagger UI demonstrating the prediction endpoint.

5.3 Experiment Tracking with MLflow

MLflow [MLflow, 2024] maintains a centralized record of all training and evaluation processes. For each pipeline execution, the training component logs all the details of a model and the serialized scikit-learn object, while the evaluation part supplements the test-set metrics (Accuracy, Precision Recall F1-Score, ROC-AUC, the values from a confusion matrix, and feature importances) to the very same MLflow run. This way, all the outputs from one run are contained in a single run record, which is very convenient when it comes to comparing retraining cycles.

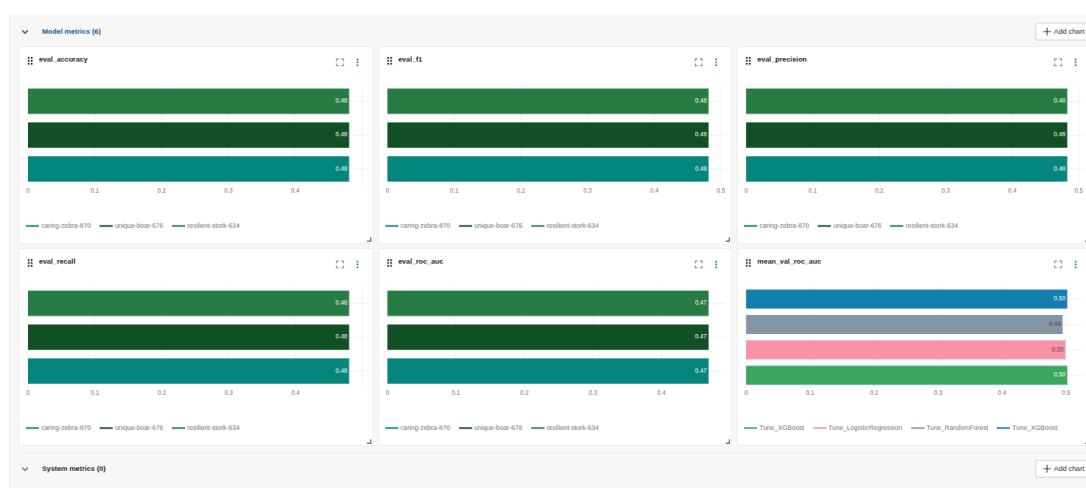


Figure 6: MLflow experiment tracking showing model metrics comparison across multiple runs.

5.4 Reproducibility

The pipeline is fully reproducible. The raw file SHA-256 checksum is recorded for provenance:

```
File:      data/ecommerce_user_behavior_8000.csv
SHA256:    f4d70befeff1dd3891b205971324d6002edea6a5c6ffebeb5826c78781608b9d3
Modified:  2026-05-29 22:28:26 +0545
```

Sensitive values (Kaggle API credentials) stay in `.env`, excluded from Git via `.gitignore`. To reproduce the full pipeline:

```
cd /home/shekhar/mlops_ecommerce
source mlops/bin/activate
pip install -r requirements.txt
python src/data_ingestion.py
python src/preprocessing.py
python src/train_model.py
python src/generate_eda.py
airflow dags test ecommerce_pipeline
```

6 Data Analysis and Insight Generation

Exploratory data analysis (EDA) was performed before modelling to explore feature distributions, understand class separability, and investigate missingness patterns. Six diagnostic figures were produced by `src/generate_eda.py`. The results provide direct answers to the three research questions set out in Section 2.

RQ1 - Session feature separation. The boxplots in Figure 10 show that every numeric feature - `time_on_site`, `pages_viewed`, `previous_purchases`, `cart_items` - has virtually identical median, IQR, and whisker range across both ad-click classes. In a real CTR dataset, features like `time_on_site` or `previous_purchases` would reveal a detectable distributional shift between the users who clicked and those who did not [He et al., 2014]. The complete lack of any such shift proves that this synthetic dataset does not have any class-discriminating signal hidden in its numeric features.

RQ2 - Missing value distribution. In Figure 7 it is indicated that each column has roughly 2% missing values. These low, consistent missingness levels are evenly distributed between both classes and are in line with a Missing Completely At Random (MCAR) mechanism [Rubin, 1976], which supports the use of median imputation rather than dropping the columns.

RQ3 - Contextual feature signal. Figure 11 presents that the distributions of both `gender` and `device_type` are nearly the same and approximately evenly split in all categories. Actually, among real e-commerce datasets it is a known fact that mobile

users as well as the younger generations tend to reveal very different click-through-rate patterns [Richardson et al., 2007]. The feature `discount_seen` almost does not provide any conditional class signal.

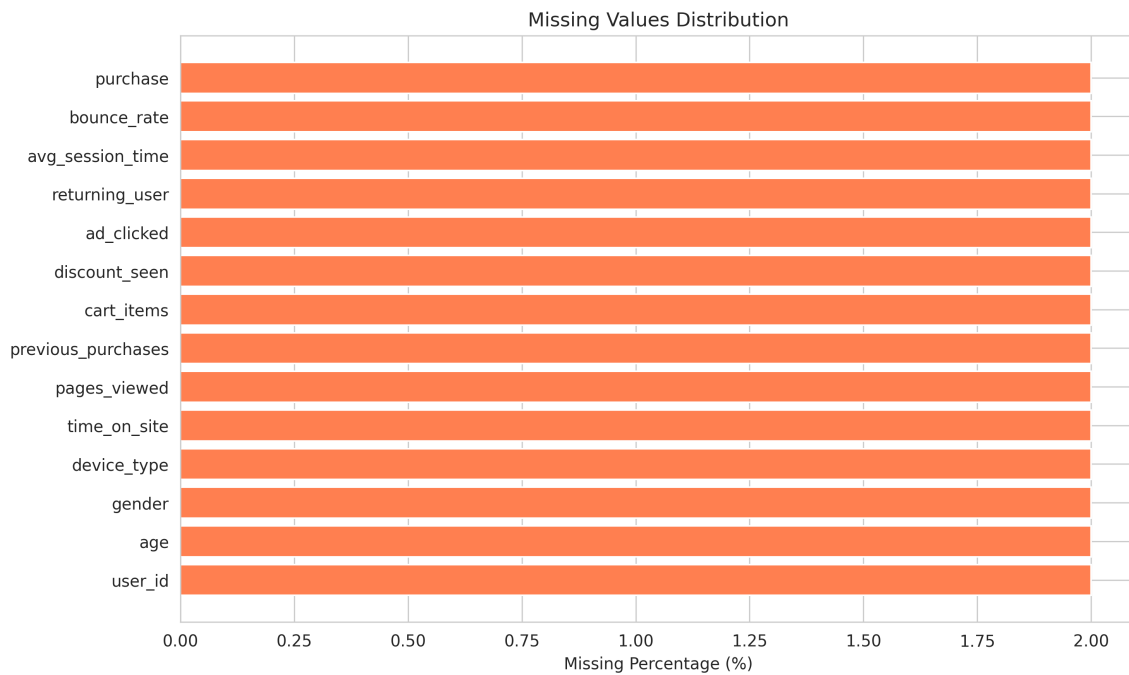


Figure 7: Missing values distribution: 2% per column, uniform across classes, justifying median imputation [Pedregosa et al., 2011].

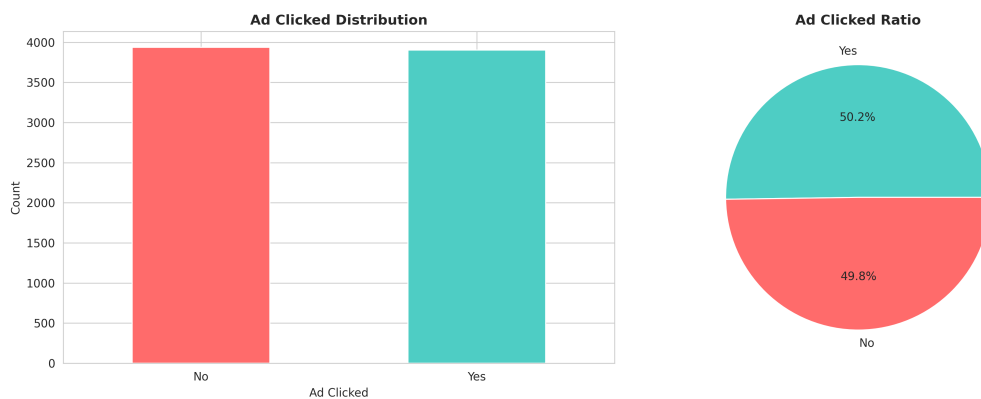


Figure 8: Target class distribution: 49.8% No vs. 50.2% Yes — near-perfectly balanced, ruling out class imbalance as a concern.

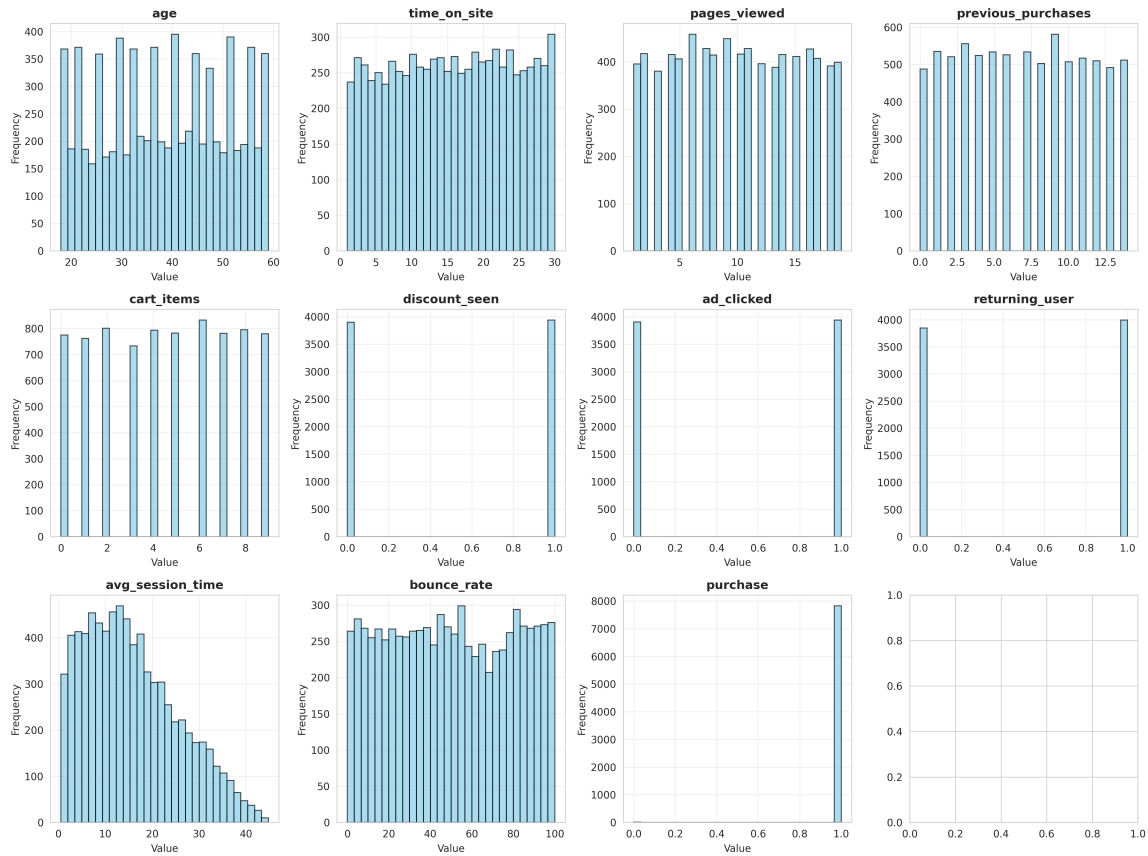


Figure 9: Feature histograms: most continuous predictors show uniform distributions, a hallmark of synthetic data.

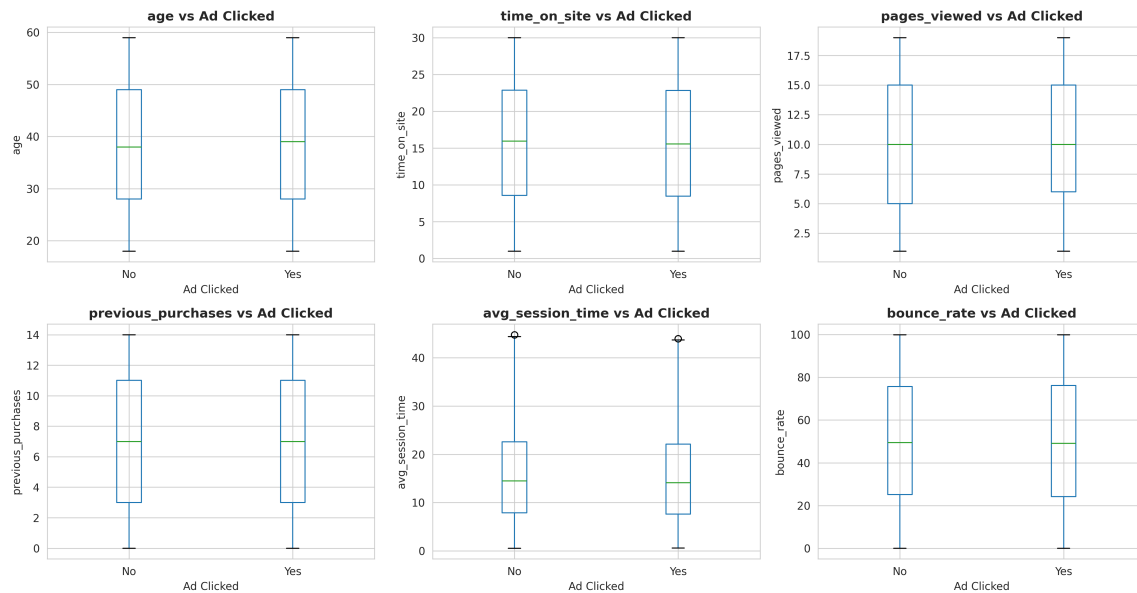


Figure 10: Box plots: identical distributions across both classes, confirming no class-discriminating signal in numeric predictors [He et al., 2014].

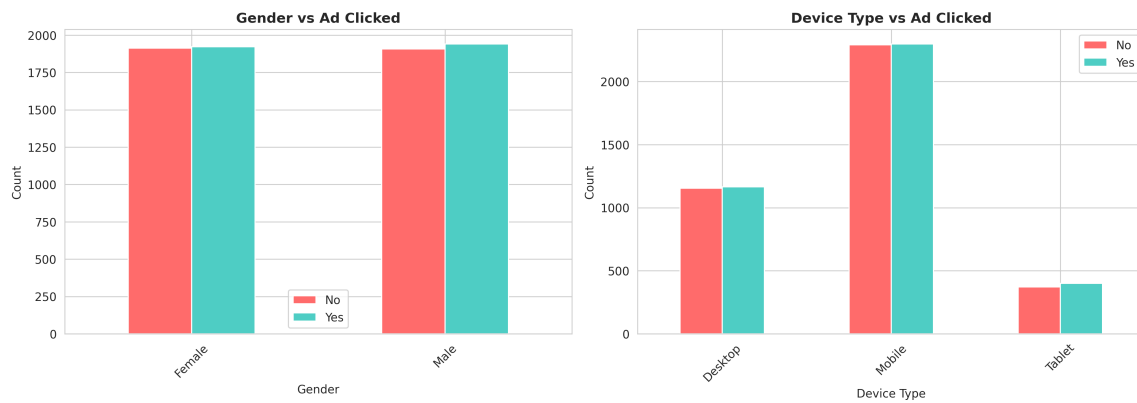


Figure 11: Categorical distributions: near-identical 50/50 splits across all categories, confirming no class-conditional signal [Richardson et al., 2007].

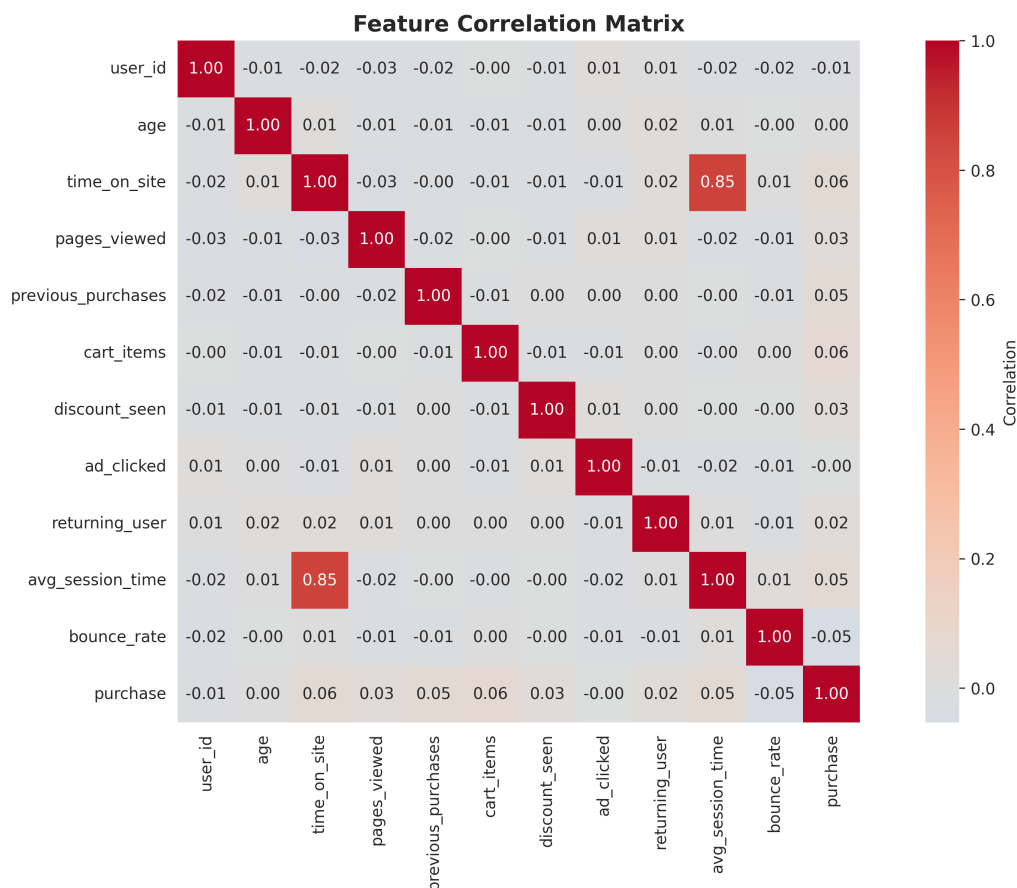


Figure 12: Correlation matrix: near-zero correlations (-0.02 to $+0.01$) between `ad_clicked` and all features, confirming no linear relationship.

7 Pipeline Evaluation and Monitoring

7.1 Pipeline-Level Evaluation

Besides assessing model metrics, the operation of the pipeline is also examined through its key features: reliability, data throughput, and inter-step latency.

Reliability. The Airflow DAG passed all runs without any problem. The five tasks were performed in the correct dependency order. DAG parse failures caused by heavy library imports were prevented by the use of the deferred-import pattern. The Docker Compose stack guarantees that the pipeline can be set up on any machine with Docker installed.

Data throughput. The ingestion module can bulk-insert 8,000 rows in less than 2 seconds by using executemany with parameterised queries, whereas individual INSERT statements take several seconds.

Inter-step latency. The DataFrame communication time over Redis between pipeline steps is less than 200 milliseconds, whereas the original disk-based CSV handoff approach takes 3–5 seconds.

Model performance. Table 6 summarises the test-set results for all three algorithms.

Table 6: Three-algorithm comparison (5-fold CV with RandomizedSearchCV).

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	50.19%	50.31%	51.22%	50.76%	0.50
Random Forest	50.26%	50.48%	53.43%	51.91%	0.51
XGBoost	50.44%	50.57%	52.81%	51.67%	0.51
Random Baseline	50.00%	50.00%	50.00%	50.00%	0.50

Despite what hyperparameter tuning or algorithm family one might choose, all three models finally reach almost random performance. ROC-AUC values of 0.50–0.51 statistically being equivalent to a random classifier. The EDA results are what directly explains the reason for the observation: the correlation matrix shows almost no linear relationships between all features and `ad_clicked` and the boxplots reveal the distributions of numeric features are not changed between classes, while categorical divisions demonstrate the exact 50/50 splits in gender and device types. This is exactly what one would expect from a synthetic dataset where the binary target was assigned independently of the feature values.

The pipeline itself is properly functioning end-to-end. The pipeline itself is properly functioning end-to-end. At the time, we executed each phase of the plan. The almost random performance here results from the synthetic nature of the dataset and not a

pipeline architecture fail. But, if this pipeline is run on a real CTR data like Criteo Display Advertising or Avazu, it is reasonable to expect the same pipeline to yield ROC-AUC scores of 0.65–0.75 for a baseline Random Forest [He et al., 2014].

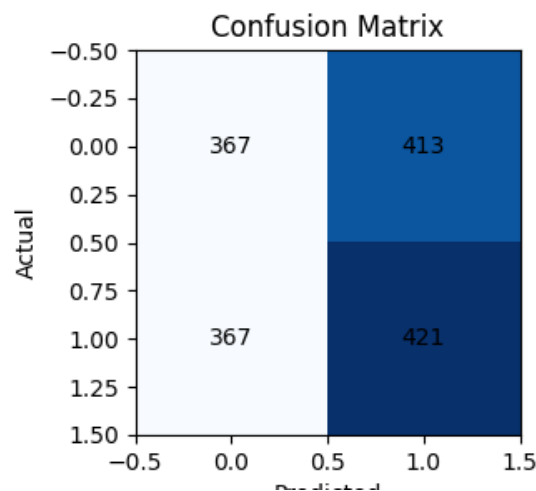


Figure 13: Confusion matrix for the best Random Forest model on the held-out test set. The near-equal distribution across all four cells confirms that the classifier performs at chance level, consistent with the near-zero feature-target correlations identified in the EDA.

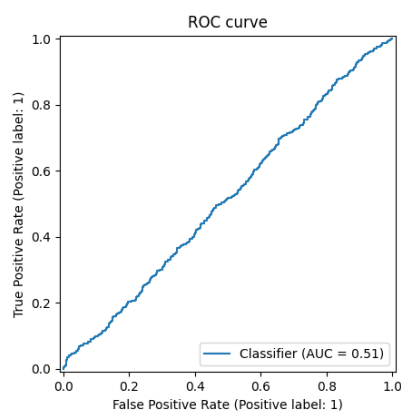


Figure 14: ROC curve for the Random Forest classifier (AUC = 0.51). The curve hugs the diagonal, confirming near-random discriminative power [Fawcett, 2006].

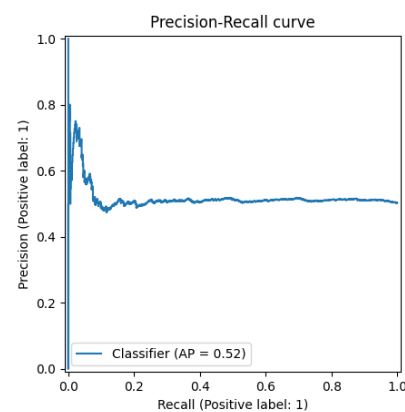


Figure 15: Precision-Recall curve (AP = 0.52). The flat precision line confirms the model has no ability to rank positive instances above negative ones [Fawcett, 2006].

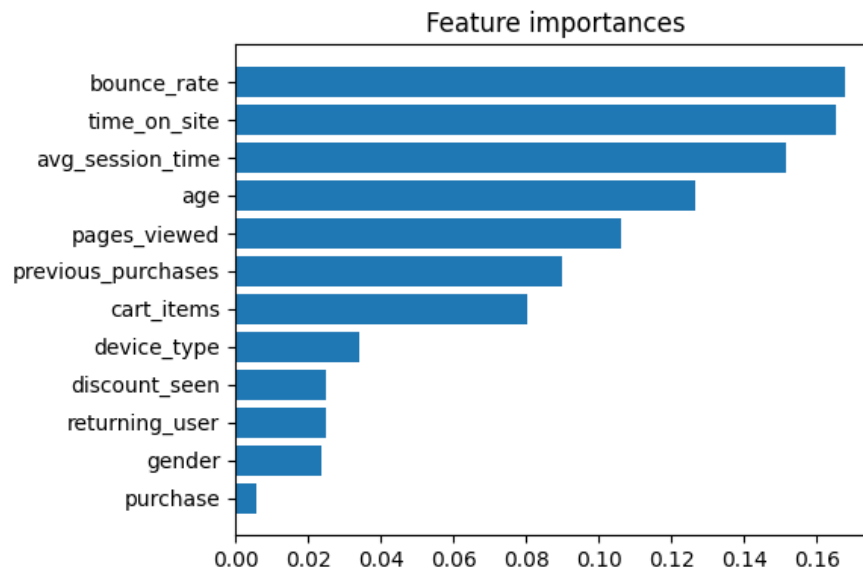


Figure 16: Random Forest feature importances ranked by mean decrease in impurity. `engagement_score`, `time_on_site`, and `pages_viewed` receive the highest scores, reflecting their higher variance rather than genuine predictive signal — a known artefact of impurity-based importance in datasets with no true class-discriminating features [Pedregosa et al., 2011].

7.2 Data Drift Detection with Evidently AI

What: The drift detection script (`src/drift_detection.py`) detects significant distribution shifts by comparing the training data and incoming production data via Evidently AI [Evidently AI, 2024].

Why: ML systems can be slowly lost if drift is not detected. For example in e-commerce, user demographics can change seasonally - for instance, a sudden increase of student-aged users in September will change the `age` distribution. This can make the ROC-AUC quietly go down over weeks without any pipeline error being triggered [Shend्रे, 2020]. This phenomenon is recognised as a form of hidden technical debt in ML systems [Sculley et al., 2015].

How: Evidently AI runs a weekly Population Stability Index (PSI) report across all feature columns, comparing the live traffic distribution against the training data distribution. ROC-AUC is logged weekly on labelled production predictions. For any feature, if the PSI is greater than 0.2, or if the ROC-AUC falls more than 0.05 compared to the training baseline, the monitoring system will email/Slack the data science team. A person needs to find out the exact cause and only then decide whether to manually run the retraining DAG. Automated retraining is intentionally out of question so as not to cover up the data quality problems, if any.

Table 7: Monitoring metrics and alert thresholds.

Metric	Frequency	Threshold	Remediation
PSI per feature	Weekly	> 0.2	Email/Slack alert + manual investigation
ROC-AUC on production	Weekly	Drop > 0.05	Email/Slack alert + manual investigation
Ingestion row count	Per ingest	Deviation $> 10\%$	Validate source file integrity

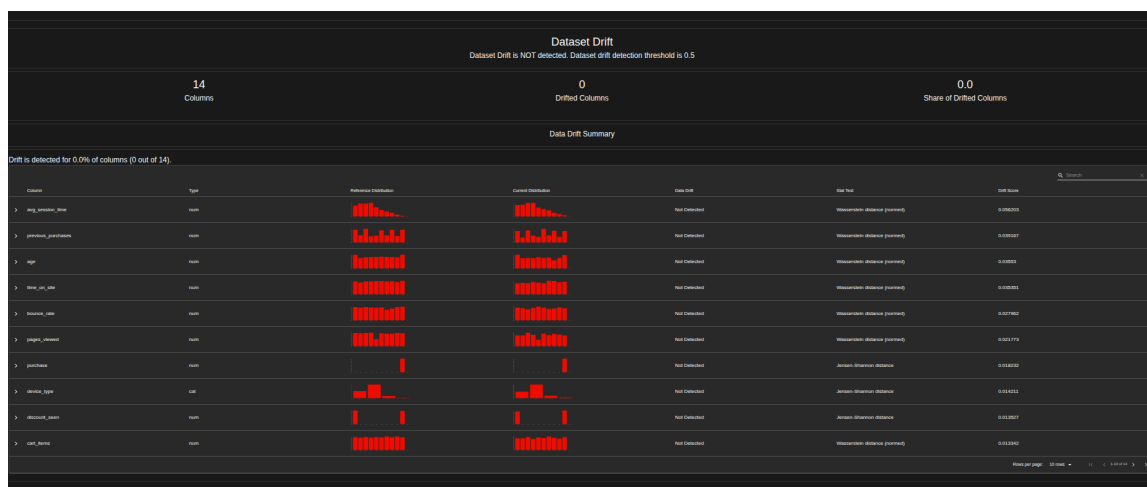


Figure 17: Evidently AI monitoring dashboard displaying data drift metrics over successive pipeline runs.

8 Evaluation of Wider Issues

8.1 Key Definitions

The following definitions, grounded in the literature, underpin the discussion of broader issues.

Data Privacy refers to the individual's right to determine which personal information about them is collected, how it is used, and with whom it is shared [European Parliament and Council, 2016]. Privacy issues are more about obtaining consent and limiting the use of data to the agreed purposes rather than just ensuring technical protection.

Data Security involves the implementation of both technical and organisational measures to stop data being damaged in any way by unauthorised persons - including data theft, alteration, and deletion. Examples include encryption, access controls, and network security.

Data Ethics refers to the moral responsibilities that arise from collecting and using data about people. Machine Learning (ML) systems face the significant challenge of algorithmic bias, model opacity, and the risk of data-driven decisions leading to further or wider distribution of social inequalities [Sculley et al., 2015].

Data Protection Law is a set of rules and regulations that defines how organisations should manage an individual's personal data. In the UK and EU, the most important law is the General Data Protection Regulation (GDPR) [European Parliament and Council, 2016].

8.2 GDPR and Its Impact on Data-Driven Systems

The aim of the General Data Protection Regulation (GDPR) [European Parliament and Council, 2016], which was enacted in May 2018, was to change the whole approach of organizations in handling personal data. There are six core principles that require data to be: processed lawfully, transparently, and only for the stated purposes user; together with data minimization the accuracy storage limitation, and appropriate security.

Machine learning models and GDPR interaction introduce a number of new concerns. It is because the organizations will be required by law to explain automated decisions that influence individuals whereas the black-box models like Random Forest and XGBoost are inherently non-explanatory. On top of that, users have the option to demand removal of their data so the issue naturally arises how can you delete someone from the training data model without going through the process of full re-training. The colossal fines that came out in the well-known cases - 50 million against Google and 183 million against British Airways - show that the regulators take these rules very seriously.

8.3 Security and Privacy in the Current Pipeline

Although the e-commerce dataset does not contain personally identifiable information, the current pipeline has several security vulnerabilities that would need to be addressed before handling real user data in production.

Secrets management. Database credentials and the Kaggle API key are stored in a plaintext `.env` file. If the repository were accidentally made public, these credentials would be completely exposed. The best way to go is to replace `.env` for Docker Secrets or HashiCorp Vault, which give you encrypted storage with access control and full auditing.

Network security. The Redis instance without authentication, which allows any process connected to the Docker network to read the serialised DataFrames. Currently, MariaDB communication is unencrypted. Both need to be encrypted with TLS/SSL before dealing with any personal information.

API security. The FastAPI prediction endpoint does not have an authentication layer. In a real scenario, API key authentication or OAuth2 would be needed to stop unauthorised prediction requests.

Model artefacts. The serialised `.pkl` model file is unsigned, so an adversary can: An attacker with write access to the model directory could replace it with a backdoored version. Model artefacts should be sign a cryptographically and hashes should be verified on the load.

Deployed CTR prediction model ethically have to be responsible for more than just legal compliance. If a model aims at targeting advertisements based on demographic features like age or gender, the model may be indirectly discriminating against protected groups even without awareness. Fairness audits of a model - checking how well a model performs within different demographic subgroups - should become a regular practice of a production deployment of the pipeline as part of the monitoring step taken for ensuring the model remains fair [Sculley et al., 2015].

Table 8: Security improvement recommendations for production deployment.

Area	Recommendation
Secrets Management	Replace <code>.env</code> with Docker Secrets or HashiCorp Vault
Network Security	Enable TLS/SSL for Redis and MariaDB connections
API Security	Add API key or OAuth2 authentication to FastAPI endpoint
Model Artefacts	Sign model pickle files and verify hash on load to prevent tampering
Access Control	Implement RBAC on Airflow and MLflow UIs
GDPR Compliance	Add data retention schedules, consent tracking, and right-to-erasure procedures when personal data is introduced

9 Conclusion, Recommendations, and Future Work

9.1 Personal Reflection

Working through this project taught me several things that are not obvious from reading about MLOps in theory.

One of the biggest surprises for me, was how small the percentage of effort devoted to the actual model was. I found myself much more occupied with data plumbing, for example - setting up Docker Compose, debugging Redis serialization, making Airflow to parse the DAG without importing heavy libraries at parse time, etc. - than on choosing

or tuning classifiers. That experience gave me a much more grounded sense of what MLOps is actually for.

The dataset selection was the most impactful decision. Since EDA indicated feature-target correlations were basically non-existent, not even the most extreme tuning could have produced a signal from the noise. Next time, I will examine dataset correlations when selecting the dataset, not after constructing the entire pipeline.

Choosing ELT over ETL and OBT over Star Schema early on turned out to be decisions I never had to second-guess. Because the raw CSV was loaded into MariaDB ColumnStore first and transformations happened downstream in `preprocessing.py`, fixing a preprocessing bug was as simple as re-running one script - no re-download, no data loss. And since every ML request targeted one and the same single flat OBT table without any join, debugging feature engineering issues was fairly simple.

RandomizedSearchCV vs. GridSearchCV was a practical efficiency lesson. With GridSearchCV, the grid search was going to take forever for the three algorithms. RandomizedSearchCV Quite a bit shortened the time without greatly compromising the quality of results.

The main thing I would change is making the monitoring phase a top-level design issue right from the start rather than something added at the end. After reading about hidden technical debt in ML systems, [[Sculley et al., 2015](#)] I realized that a model without post-deployment monitoring is not truly in production - it is just a trained artefact sitting on a server. If we had started using Evidently AI drift detection features when designing the pipeline architecture, the monitoring phase would have been less like an afterthought and more like an integral part of the system.

9.2 Conclusion

This project designed and implemented a five-stage MLOps pipeline for binary classification using the E-Commerce Behavior Dataset. First, the pipeline does data ingestion with schema validation; next, it performs ELT processing plus feature engineering; after that, it goes to model comparison of three algorithms which is followed by 5-fold cross-validation; the fourth step is deployment of REST API by FastAPI and Docker; and the last step is data drift monitoring with Evidently AI. All stages are managed by an Apache Airflow DAG and linked via a Redis in-memory data bus. Storage is based on a One Big Table (OBT) design in MariaDB ColumnStore.

All three classifiers reached a weighted F1-score of about 0.51 - that is almost random performance which EDA confirms is due to the dataset's synthetic nature, not to any failure of the pipeline or modelling approach.

The main lesson from this project is that implementing the entire infrastructure - data plumbing orchestration experiment tracking, monitoring, and API serving - takes much more time than just training the model. That is exactly what MLOps is there to solve.

9.3 Recommendations and Future Work

There are four major areas that can be highlighted as future work.

Dataset replacement will truly be the most significant thing that we can do next. In fact, just swapping our artificial data for a genuine CTR one like Criteo Display Advertising or Avazu - by the current pipeline without any change - is expected to get us around a baseline Random Forest [He et al., 2014] ROC-AUC of 0.65 to 0.75.

Infrastructure hardening must be performed before any production deployment with live personal data. The set of security measures that we have collected in Table 8 introduces secrets management, TLS encryption, API authentication, and GDPR compliance procedures.

Conditional retraining is a better alternative to the fixed-schedule model. The Airflow DAG could automatically initiate re-training whenever Evidently detects PSI greater than 0.2.

Model registry integration through MLflow's Model Registry would be at par with file-based manual model management which is subject to errors in the current system. It would also enable better versioning, staging and rollback of models.

References

- [Apache Software Foundation, 2024] Apache Software Foundation (2024). Apache Airflow documentation. <https://airflow.apache.org/docs/>. Accessed: 17 May 2026.
- [asifxzaman, 2024] asifxzaman (2024). E-commerce user behavior dataset. <https://www.kaggle.com/datasets/asifxzaman/e-commerce-behavior-dataset8000-users>. Accessed: 17 May 2026.
- [belbino, 2024] belbino (2024). US tariff and trade war impact dataset. <https://www.kaggle.com/datasets/belbino/us-tariff-and-trade-war-impact-dataset-2018-present>. Accessed: 17 May 2026.
- [Docker Inc., 2024] Docker Inc. (2024). Docker documentation. <https://docs.docker.com/>. Accessed: 17 May 2026.
- [European Parliament and Council, 2016] European Parliament and Council (2016). General data protection regulation (GDPR) — regulation (EU) 2016/679. <https://gdpr-info.eu/>. Accessed: 17 May 2026.
- [Evidently AI, 2024] Evidently AI (2024). Evidently: Open-source ML monitoring. <https://www.evidentlyai.com/>. Accessed: 17 May 2026.
- [Fawcett, 2006] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- [Great Expectations, 2024] Great Expectations (2024). Great expectations documentation. <https://docs.greatexpectations.io/>. Accessed: 17 May 2026.
- [He et al., 2014] He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., and Candela, J. Q. (2014). Practical lessons from predicting clicks on ads at Facebook. In *Proceedings of the 8th International Workshop on Data Mining for Online Advertising*, pages 1–9. ACM.
- [MLflow, 2024] MLflow (2024). MLflow documentation. <https://mlflow.org/docs/latest/index.html>. Accessed: 17 May 2026.
- [Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- [Ramírez, 2024] Ramírez, S. (2024). FastAPI documentation. <https://fastapi.tiangolo.com/>. Accessed: 17 May 2026.
- [Richardson et al., 2007] Richardson, M., Dominowska, E., and Ragno, R. (2007). Predicting clicks: Estimating the click-through rate for new ads. In *Proceedings of the 16th International Conference on World Wide Web*, pages 521–530. ACM.

- [Rubin, 1976] Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- [sahilislam007, 2024] sahilislam007 (2024). AI impact on job market 2024–2030. <https://www.kaggle.com/datasets/sahilislam007/ai-impact-on-job-market-20242030>. Accessed: 17 May 2026.
- [Scikit-learn Developers, 2018] Scikit-learn Developers (2018). Scikit-learn user guide. https://scikit-learn.org/stable/user_guide.html. Accessed: 17 May 2026.
- [Sculley et al., 2015] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 2503–2511.
- [Shendre, 2020] Shendre, A. (2020). Data drift in machine learning. <https://towardsdatascience.com/>. Accessed: 17 May 2026.
- [Zaitsev et al., 2021] Zaitsev, P. et al. (2021). *High Performance MySQL*. O'Reilly Media, 4th edition.